Содержание

Введение 4
1 Структура и общая технологическая схема производства, характеристики основных видов продукции 6
2 Описание структуры ВЦ, АСУП и производственного отдела, где проходит практика 7
3 Должностные обязанности оператора ЭВМ, техника-программиста, инженера-программиста 9
4 Анализ задачи 11
4.1 Анализ предметной области 11
4.2 Инструменты разработки 13
4.3 Диаграмма вариантов использования 13
4.4 Постановка задачи 15
5 Проектирование задачи 16
6 Реализация 17
7 Тестирование 21
8 Применение 22
Заключение 29
Список использованных источников 30
Приложение А Листинг программы
Приложение Б Диаграммы UML
Приложение В Тесты на использование

Введение

На преддипломную практику поставлена задача разработки игрового приложения «Career start».

Цель проекта – разработать интерактивную 2D-игру обучающе-профориентационного характера, которая моделирует путь от студента до трудоустройства выпускника Гродненского государственного политехнического колледжа.

Создаваемая игра рассчитана в первую очередь, на учащихся и выпускников Гродненского государственного политехнического колледжа, а также всех, кто интересуется темой профессионального становления. Игровой процесс направлен на симуляцию ключевых этапов подготовки к карьере, интерактивное обучение основам учебного процесса, формирование адаптивной системы мотивации и оценки, предоставление удобного справочного инструментария и обеспечение персонализированного опыта для игрока.

Ниже приведено краткое описание разделов пояснительной записки.

Первый раздел носит название «Структура и общая технологическая схема производства, характеристики основных видов продукции». В этом разделе описываются основные производственные процессы компании ООО «Сенла групп», специфика предоставляемых услуг, а также её организационная структура. Здесь представлено общее описание работы компании, на базе которой проходила практика.

Второй раздел – «Описание структуры ВЦ, АСУП и производственного отдела, где проходит практика». В этом разделе рассматривается структура вычислительного центра, автоматизированной системы управления предприятием и производственного отдела компании ООО «Сенла групп».

Третий раздел – «Должностные обязанности оператора ЭВМ, техника-программиста, инженера-программиста». В нем приводится описание обязанностей сотрудников, участвующих в работе вычислительного центра, а также их роль в процессе разработки программного обеспечения.

Четвертый раздел носит название «Анализ задачи». В этом разделе можно ознакомиться с анализом предметной области поставленной задачи. В подразделе «Инструменты разработки» описана среда разработки, используемая для реализации проекта, и установлены минимальные и оптимальные требования к аппаратным характеристикам для его правильного функционирования. Далее приведена диаграмма вариантов использования, которая отражает взаимодействие пользователя с системой. В завершении раздела сформулирована постановка задачи, где детализированы требования и цели проекта.

В разделе «Проектирование задачи» рассмотрены основные аспекты проектирования игрового приложения. Раздел включает диаграмму деятельности.

«Реализация» – шестой раздел пояснительной записки, в котором представлено описание основных классов игрового приложения «Career start».

Седьмой раздел – «Тестирование». В данном разделе приведен отчет о тестировании проекта.

В разделе «Применение» описано назначение, область применения и среда функционирования игрового приложения. Также включены инструкции по инсталляции и выполнению программы.

«Заключение» содержит краткую формулировку задачи, результаты проделанной работы, описание использованных методов и средств.

В разделе «Список использованных источников» представлен перечень литературы, справочных материалов и источников, использованных при выполнении проекта.

В приложении А к пояснительной записке представлен листинг программы.

В приложении Б представлены диаграммы UML, которые иллюстрируют архитектуру и основные аспекты реализации программного обеспечения.

В приложении В представлены тесты на использование.

1 Структура и общая технологическая схема производства, характеристики основных видов продукции

ООО «Сенла груп» – международная IT-компания, специализирующаяся на заказной разработке программного обеспечения полного цикла.

Производственная структура ООО «Сенла груп» организована по матричному типу. «Производственными цехами» здесь выступают технологические департаменты (Competence Centers), разделенные по стекам технологий. Основными элементами структуры являются:

– технологические департаменты: Java, Frontend, Mobile, Salesforce, QA (тестирование), BA (бизнес-анализ);

– проектные команды: временные объединения специалистов из разных департаментов для работы над конкретным заказом;

– отдел проектного управления: обеспечивает координацию между заказчиком и разработчиками.

Процесс создания программного продукта в компании строится на базе гибких методологий (Agile/Scrum) и включает следующие технологические этапы:

– анализ и спецификация: определение требований ПО бизнес-аналитиками;

– проектирование: разработка архитектуры системы, выбор баз данных и связей между модулями;

– разработка: написание программного кода на языке Java с использованием фреймворка Spring. Процесс включает Code Review (взаимную проверку кода);

– тестирование: проверка продукта на соответствие требованиям (мануальное и автоматизированное тестирование);

– сборка и развертывание: автоматизированная доставка кода на тестовые и рабочие серверы;

Продукцией компании является интеллектуальная собственность в виде программного обеспечения. Основные виды:

– Enterprise-системы: крупные корпоративные платформы для автоматизации бизнес-процессов.

– FinTech-решения: банковские системы, платежные сервисы, платформы цифрового банкинга;

– MedTech-проекты: информационные системы для медицинских учреждений и аптечных сетей;

– Salesforce-решения: индивидуальная настройка и разработка на базе платформы Salesforce.

Ключевые характеристики продукции: Высокая степень информационной безопасности, масштабируемость, кроссплатформенность и высокая

2 Описание структуры ВЦ, АСУП и производственного отдела, где проходит практика

Роль классического ВЦ в компании выполняет «Отдел системного администрирования и DevOps», который обеспечивает техническую базу для производства:

– аппаратное обеспечение: локальная сеть офиса в Гродно, серверное оборудование для внутреннего тестирования, системы бесперебойного питания и хранения данных;

– облачная инфраструктура: использование мощностей AWS (Amazon Web Services), Azure и Google Cloud для развертывания (Deployment) разрабатываемых продуктов;

– сетевая безопасность: настройка VPN-каналов для безопасного доступа к ресурсам заказчиков, управление правами доступа и антивирусная защита;

– инструменты CI/CD: поддержка серверов автоматической сборки (Jenkins, GitLab CI), что позволяет автоматизировать процесс «производства» ПО от написания кода до его тестирования.

В ООО «СЕНЛА ГРУП» функции АСУП выполняет комплекс интегрированных облачных и внутренних решений, обеспечивающих автоматизацию бизнес-процессов:

– система управления проектами и задачами (Jira): используется для планирования спринтов, распределения задач между разработчиками и контроля сроков исполнения;

– база знаний (Confluence): централизованное хранилище технической документации, регламентов компании и проектных спецификаций;

– система управления ресурсами (RM-система): собственная внутренняя разработка компании для учета рабочего времени (Logging time), управления отпусками и оценки эффективности сотрудников (KPI);

– HR-платформа: для автоматизации процессов рекрутинга и адаптации новых сотрудников.

Местом прохождения практики являлся производственный отдел – Департамент Java-разработки. Его структура в гродненском филиале включает:

– руководитель департамента (Head of Java Unit): осуществляет общее управление ресурсами, контроль квалификации сотрудников и распределение специалистов по проектам;

– Resource Managers: занимаются карьерным развитием разработчиков и мониторингом их загрузки;

– проектные группы (Project Teams): мобильные звенья, состоящие из Java-разработчиков (уровней Junior, Middle, Senior), которые непосредственно работают над кодом заказчика.

Функции отдела:

– проектирование серверной части (Backend) высоконагруженных систем;

– работа с современным стеком технологий: Spring Framework (Boot, Security, Data), Hibernate, архитектура микросервисов;

– интеграция приложений с базами данных (PostgreSQL, Oracle) и сторонними сервисами через API.

3 Должностные обязанности оператора ЭВМ, техника-программиста, инженера-программиста

В ООО «Сенла груп» распределение обязанностей между специалистами основывается на уровне их квалификации, опыте и сложности решаемых задач. Несмотря на современные названия должностей в индустрии (Junior/Middle/Senior Developer), их функции соответствуют классическим квалификационным категориям.

Оператор ЭВМ: в современной IT-среде эта роль соответствует специалистам начального уровня поддержки или системного администрирования.

Основные обязанности оператора ЭВМ включают в себя следующее:

– обеспечение бесперебойной работы компьютерной техники, периферийных устройств и сетевого оборудования в офисе;

– установка, настройка и обновление общесистемного и прикладного программного обеспечения (ОС, драйверы, браузеры);

– ввод и обработка технической информации в корпоративных системах (Jira, Confluence);

– техническое сопровождение процессов резервного копирования данных.

Техник-программист: соответствует уровню Junior Java Developer. Специалист, выполняющий практические задачи под руководством наставника.

Основные обязанности техника-программиста включают в себя следующее:

– написание программного кода на языке Java для отдельных модулей системы по детальному техническому заданию;

– проведение модульного тестирования (Unit testing) собственного кода для выявления синтаксических и логических ошибок;

– участие в процессе исправления дефектов (Bug fixing), выявленных отделом тестирования;

– изучение и строгое соблюдение корпоративного стандарта написания кода (Code Convention);

– ведение технической документации на разрабатываемые фрагменты программного обеспечения.

Инженер-программист: соответствует уровню Middle/Senior Java Developer. Специалист, принимающий архитектурные решения.

Основные обязанности инженера-программиста включают в себя следующее:

– проектирование архитектуры программных комплексов, разработка структур баз данных и логики взаимодействия микросервисов;

– разработка сложной бизнес-логики серверной части приложений с использованием стека технологий Java (Spring Boot, Hibernate и др.);

– проведение Code Review – анализа и проверки качества кода, написанного техниками-программистами;

– анализ требований заказчика и их декомпозиция на конкретные технические задачи для команды;

– оптимизация производительности приложений и обеспечение безопасности обрабатываемых данных;

– настройка инструментов автоматизации разработки и деплоя (CI/CD).

Так как моя практика проходила по направлению Java, в данном разделе акцент сделан именно на разработку серверной части, что соответствует специфике моего отдела в ООО «Сенла груп».

4 Анализ задачи

4.1 Анализ предметной области

Предметной областью данного проекта является сегмент образовательных компьютерных игр и симуляторов карьеры. Это направление сочетает в себе методы игрового дизайна для вовлечения пользователя с решением конкретных учебных или профориентационных задач. В отличие от развлекательных игр, ключевым показателем эффективности здесь является не только игровая удовлетворённость, но и достижение образовательного результата: усвоение информации, формирование навыков или изменение отношения к предмету.

В рамках жанра можно выделить несколько ключевых видов:

– «чистые» тренажёры: фокус на отработке узкого навыка;

– приключенческие игры с обучающими элементами: сюжетная игра, где прогресс напрямую связан с решением учебных задач (например, «Квесты» по физике или истории);

– симуляторы процессов: игры, моделирующие определённую деятельность (управление городом, предприятием, карьерой — например, «Sims», «Карьера»);

– интерактивные викторины и головоломки: игровая механика построена вокруг вопросов и заданий (пазлы, тесты, ребусы).

Игра «Career Start» относится к гибридному виду, сочетающему в себе элементы симулятора карьеры (моделирование пути выпускника) и приключенческой игры с обучающими головоломками (мини-игры на знание документов и процессов колледжа).

Программа призвана автоматизировать и модифицировать следующие задачи: интерактивное представление учебного материала, моделирование последствий, персонализация обучения, управление игровым прогрессом.

Обработка данных в реальном времени и создание интерактивной среды необходимы по следующим причинам:

– обратная связь: только компьютерная программа может мгновенно реагировать на действия игрока, показывая процент выполнения, выдавая фрагмент диплома нужного качества и предоставляя подсказки;

– геймификация: ручное моделирование системы мотивации (звёзды, прогресс) и случайных элементов невозможно. Компьютерная реализация делает процесс объективным, измеримым и вовлекающим;

– иммерсивность (погружение): Создание целостного игрового мира с комнатой, персонажем, интерфейсом телефона возможно только в цифровой интерактивной среде;

– масштабируемость: компьютерная основа позволяет в будущем легко добавлять новые задания, сюжетные ветки и механики без переработки всей системы.

В настоящее время профориентационная подготовка учащихся колледжа решается преимущественно традиционными, неигровыми методами: лекции и инструктажи, бумажные бланки и схемы, текстовые памятки и PDF-документы. Данные методы часто являются скучными для цифрового поколения, не обеспечивают глубокого вовлечения и не дают мгновенной, наглядной обратной связи о качестве усвоения знаний.

Период использования данной программы не ограничен.

Разрабатываемый программный продукт должен кардинально упростить и улучшить решение образовательной задачи:

– вместо пассивного изучения – активное взаимодействие: игрок не читает о схеме практик, а собирает её;

– вместо абстрактных последствий ошибок – наглядная визуализация: ошибки приводят не к абстрактной низкой оценке, а к «пятнам на дипломе», что создаёт более личную и понятную мотивацию;

– вместо разрозненных источников информации – единая интерактивная среда: все необходимые данные (правила, справочники, задания) интегрированы в игровой мир.

– вместо стандартного подхода – персонализированный опыт: учёт особенностей персонажа с самого начала создаёт ощущение индивидуального пути.

Прямых аналогов, посвящённых именно Гродненскому политехническому колледжу, не существует. Однако на рынке представлены игры смежных жанров, одним из ярких примеров является игра «Sims 4: Университет»: жизненный симулятор, где есть элементы учёбы в вузе, но отсутствует профориентационная составляющая и конкретные образовательные задачи по документообороту. Акцент сделан на развлечении и социальном взаимодействии.

Разрабатываемая игра «Career Start» должна занять уникальную нишу, сочетая гиперлокальную тематику (конкретное учебное заведение), глубокую симуляцию учебного процесса и качественную геймификацию в формате 2D-приключения. Это позволяет решить профориентационную задачу более эффективно по сравнению с традиционными методами и существующими развлекательными аналогами.

4.2 Инструменты разработки

Для разработки данного проекта была выбрана среда Unity, которая является межплатформенной средой для создания и разработки видеоигр и приложений, разработанной компанией Unity Technologies.

Разработка будет производиться на языке программирования C# – многофункциональный язык программирования, активно развивающийся на данный момент. Он часто используется как первый язык программирования начинающих программистов, так и для реализации крупных коммерческих проектов.

Иные инструменты, используемые при разработке и создании сопутствующей документации:

– WEB-ресурс DRAW.IO – будет использоваться для создания графической части и разработки UML-диаграмм;

– Microsoft Office Word 2016 – для написания документации к программному продукту;

– Smart Install Maker – утилита для создания инсталляторов;

– Dr.Explain – инструмент разработки пользовательской документации;

Разработка проекта будет происходить на компьютере со следующими параметрами:

– процессор 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz (2.69 GHz);

– ОЗУ: 16Gb;

– память: HDD 477Gb;

– ОС: Windows 11 Pro.

4.3 Диаграмма вариантов использования

Диаграмма вариантов использования – диаграмма, отражающая отношения между актерами и прецедентами и являющаяся составной частью модели прецедентов, позволяющей описать систему на концептуальном уровне.

Суть данной диаграммы состоит в следующем: проектируемая система представляется в виде множества сущностей или актеров, взаимодействующих с системой с помощью так называемых вариантов использования.

Далее описаны основные элементы диаграммы вариантов использования.

Актёр (actor) – это внешняя сущность (человек, другая система, роль), взаимодействующая с системой. Данный элемент изображается в виде фигурки человечка, которая может быть вне рамки системы.

Вариант использования является стандартным языком UML и применяется для спецификаций общих особенностей системы и любой другой сущности.

Отдельные варианты использования обозначаются на диаграмме эллипсом, в котором содержится его краткое название.

Отношение ассоциации является главным понятием языка UML и используется при построении всех графических моделей. Оно служит для обозначения роли актера в отдельном варианте использования. На диаграмме отношение ассоциации обозначается сплошной линией между актером и вариантом использования.

Для отображения взаимосвязи экземпляров отдельного варианта использования с более общим вариантом, используется отношение расширения, обозначаемое направленный пунктирной линией со стрелкой от исходного варианта. Данная линия помечается ключевым словом «extend».

Отношение включения между двумя вариантами использования указывает, что некоторое заданное поведение для одного варианта использования включается в качестве составного компонента в последовательность поведения другого варианта использования. Данная линия помечается ключевым словом «include».

Определяя для выбранного актера варианты использования и устанавливая отношения между вариантами использования, получим полную диаграмму вариантов использования.

В данной диаграмме вариантов использования актером является игрок.

Игроку доступны следующие варианты использования: играть, читать справку, изменить настройки музыки, выйти из игры, настроить персонажа, сбросить прогресс.

У варианта использования «Играть» наблюдаются следующие «extend» связи: пролистать диалог, перейти к выполнению заданий, перемещаться по локации, посмотреть телефон, перемещаться между локациями, посмотреть инвентарь, купить игровой предмет.

У варианта использования «Изменить настройки музыки» наблюдаются следующие «extend»-связи: выбрать мелодию, остановить воспроизведение музыки, продолжить воспроизведение музыки.

У варианта использования «Настроить персонажа» наблюдаются следующие «incklude»-связи: выбрать персонажа, заполнить форму. А также «extend»-связь: изменить внешность персонажа.

У варианта использования «Посмотреть телефон» наблюдаются следующие «extend»-связи: посмотреть документы, посмотреть карту.

У варианта использования «Перейти к выполнению заданий» наблюдаются следующие «include»-связи: разместить блоки схемы, ответить на вопросы, сменит одежду персонажа, собрать предметы, найти место работы, взаимодействовать с NPC, заполнить документы.

У варианта использования «Взаимодействовать с NPC» наблюдаются следующие «include»-связи: принять задание, выбрать реплику.

Диаграмма вариантов использования представлена в приложении Б на рисунке Б.1.

4.4 Постановка задачи

Необходимо разработать образовательно-профориентационное игровое приложение под названием «Career Start». Основная цель разработки – создание многофункционального 2D-симулятора карьеры, проводящего игрока через полный цикл профессионального становления: от студента до успешно специалиста.

Игра предполагает нелинейный сюжет, разделённый на ключевые главы. Игрок начинает как студент, которому необходимо освоить основы учебного процесса через серию обучающих мини-игр. Качество их выполнения напрямую влияет на состояние итогового диплома. Этот диплом становится главным системообразующим элементом, определяющим дальнейшее развитие сюжета: чем он «чище», тем шире выбор потенциальных компаний для отработки и стажировки.

После обучающего блока игрок перейдёт к этапу «Поиска работы». Далее следует глава «Собеседование», моделирующая процесс трудоустройства через диалоги, тесты и принятие решений. Успех на собеседовании открывает финальную главу «Оформление документов и первый рабочий день», где необходимо корректно собрать пакет документов и выполнить первое задание на рабочем месте.

Разрабатываемое приложение должно поддерживать высокий уровень реиграбельности и персонализации благодаря системе ветвящегося сюжета, где каждый ключевой выбор игрока (качество обучения, выбор компании, ответы на собеседовании) приводит к различным концовкам и итоговым рабочим позициям. Игровой процесс должен являться комплексным и включать: освоение теоретического материала в интерактивной форме, принятие стратегических карьерных решений, прохождение собеседований и выполнение рабочих задач.

Особое внимание будет уделено созданию целостной, детализированной игровой среды. Интерфейс должен быть интуитивным и информативным, предоставляя игроку чёткое понимание его текущего прогресса, состояния диплома, доступных вариантов выбора и последствий принятых решений.

В завершении игры пользователь будет получать обратную связь в виде трудоустройства на должность с соответствующим уровнем дохода и удовлетворённости, что наглядно демонстрирует причинно-следственную связь между усилиями в обучении и результатом. Игровое приложение будет рассчитано на молодых людей, стоящих перед выбором профессии, и обеспечит им глубокий и мотивирующий игровой опыт, симулирующий реальный путь от учёбы к работе.

5 Проектирование задачи

Диаграмма деятельности – это поведенческая диаграмма, которая показывает поток работы или действий в рамках системы или процесса. Она отображает последовательность шагов и возможные варианты выполнения работы, включая параллельные процессы и ветвления. Диаграмма деятельности включает в себя такие элементы, как начальные и конечные узлы, узлы действий, узлы решений, вилки и слияния, а также потоки управления, которые связывают эти узлы.

Для визуализации ключевого игрового цикла проекта «Career Start» была разработана диаграмма деятельности. Она описывает фрагмент процесса прохождения первого этапа игры — от старта до получения фрагмента диплома. Данная диаграмма позволяет проиллюстрировать последовательность взаимодействий игрока с системой и условия перехода между действиями.

Диаграмма деятельности представлена в приложении Б на рисунке Б.2.

6 Реализация

Диаграмма классов – это визуальное представление структуры системы, отображающее классы, их атрибуты, методы и взаимосвязи между ними. Она является одним из ключевых элементов объектно-ориентированного проектирования и используется для моделирования статической структуры приложения.

Основная цель диаграммы классов – показать, как элементы системы связаны друг с другом, какие свойства и функции они имеют, а также какие отношения существуют между ними.

Диаграмма помогает разработчикам и архитекторам понять архитектуру системы, определить связи между объектами и грамотно спроектировать базу данных или архитектуру приложения.

Игра «Career Start» содержит 26 классов, каждый из которых имеет свои атрибуты и методы, а также выполняет свое действие.

Основные классы представлены в таблице 1.

Таблица 1 – Основные классы игрового приложения «Career Start»

Имя класса	Атрибуты	Методы	Действие
1	2	3	4
DialogManager	public Text dialogueText; public Text nameText; public Animator boxAnim; public int x; private Queue<string> sentences; private int field;	public void DisplayNextSentence() public void EndDialogue() public void Method() private void Start() public void StartDialogue(Dialogue dialogue) IEnumerator TypeSentence(string sentence)	Показывает текст диалогов
DialogTrigger	public Dialogue dialogue; private int field;	public void TriggerDialogue() public void Method()	Начинает диалог при событии
MusicSelection UI	private MusicManager musicManager; private int field;	private void Start() public void SelectMusic(int index) public void TogglePause() public void Method()	Отвечает за кнопки управления музыкой

Продолжение таблицы 1

1	2	3	4
MusicManager	public AudioClip[] musicClips; public AudioSource audioSource; private int field;	private void Awake() public void PlayMusic(int index) public void SetVolume(float volume) public void TogglePause() public void Method()	Отвечает за воспроизведение музыки
Draggable1	private Canvas canvas; private RectTransform rectTransform; private Vector3 originalPosition; private Vector2 offset; private int field;	void Awake() public void OnBeginDrag(PointerEventData eventData) public void OnDrag(PointerEventData eventData) ublic Vector3 GetCurrentPosition() public void Method()	Отвечает за перетаскивание объектов
Slot	public float targetX; public float targetY; public string y; public int x; private GameObject objectInSlot; private int field;	private void OnTriggerEnter2D(Collider2D collision) void Update() private bool IsCursorOverSlot() private void ReturnObjectToStartPosition() private void OnTriggerExit2D(Collider2D collision) public void Method()	Отвечает за приём объектов
Zadanije1	public Text[] practika_name; public Text[] otvet; public Image[] img; private int ball=0; private int plus; public Image result; public Text percent; public Text plus_star; public Button otv; public Button dia; public Image dip1; public Animator targetAnimator; public int anim_; private int field;	void Start() public void Btn_otvet_clik() public void Btn_dalee_clik() public void Method()	Считает баллы за задание 1

Продолжение таблицы 1

1	2	3	4
Zadanije1_2	public Image zd1_2; public Button otv1_2; public Button dia1_2; public Button dalee_fon1_2; public Button dalee_fon; public Image dip1; public Image dip2; public Button dalee_dip; public Animator targetAnimator2; public int anim_2; public Image result; public Text percent; public Text plus_star; private int plus; public Animator targetAnimator; public int anim_; public Image[] puzzle_image_mas; private int field;	private void Start() void Update() public void Btn_otvet_clik1_2() public void Btn_dalee_zd1_2_clik() public void Btn_dalee_clik_end() public void Method()	Считает баллы за задание 2
Dialogue	public string name; public string[] sentences; private int field;	public void Method()	Хранит реплики диалога
Room	[SerializeField] private GameObject devuchka; [SerializeField] private Image paren; [SerializeField] private Button[] bt_zadanije; [SerializeField] private Button[] bt_dia; private int field;	void Update() public void bt_start_dlg_click() public void Method()	Переключает сцены и кнопки
RadioButtonGroup	public Toggle[] radioToggles; public Text[] radio_text;	public int Property private void Start() private void OnRadioToggleValueChanged(int changedIndex) public void Method()	Отвечает за радио-кнопки, один выбор из нескольких
Forma	ublic GameObject form; public int x = 0; public InputField name_pers; public Toggle myToggle; public Image blok; private int field;	public void Btn_form_clik(GameObject clickedButton) public void Method()	Сохраняет данные игрока

Продолжение таблицы 1

1	2	3	4
Pers_vbor	[Header("First Image")] [SerializeField] private Image devuchka; [SerializeField] private Sprite devuchka_sprite; [Header("Second Image")] [SerializeField] private Image paren; [SerializeField] private Sprite paren_sprite; [Header("Click Settings")] [SerializeField] private Vector2 targetPosition = new Vector2(0, 0); private Vector2 devuchkaOriginalSize, parenOriginalSize; private Vector2 devuchkaOriginalPosition, parenOriginalPosition; private Sprite devuchkaOriginalSprite, parenOriginalSprite; private bool isSelected = false; private int field;	void Start() void Update() public void OnImageClick(Image clickedImage) public void OnPointerEnter(PointerEventData eventData) public void OnPointerExit(PointerEventData eventData) public void Method()	Выбирает персонажа
playerControler	public float speed; public float jumpForce; public float moveInput; private Rigidbody2D rb; private bool facingRight=true; private bool isGrounded; public Transform freePos; public float checkRadius; public LayerMask whatIsGround; private Animator anim; private int field;	private void Start() private void FixedUpdate() private void Update() void Flip() public void Method()	Отвечает за основное передвижение игрока.
Restsrt	private int field;	public void bt_restart_click() public void Method()	Отвечает за сброс игрового прогресса
LaunchingDocument	public string fileName; private int field; private int field1;	public void OpenDocument() public void Method()	Отвечает за открытие документа

Диаграмма классов представлена в приложении Б на рисунке Б.3.

7 Тестирование

При разработке данной программы многие возникающие ошибки и недоработки были исправлены на этапе реализации проекта. После завершения испытания реализации программы было проведено тщательное функциональное тестирование. Функциональное тестирование должно гарантировать работу всех элементов программы в автономном режиме.

Разработанные тест-кейсы представлены в приложении В.

Расписание проведения тестирования представлено в таблице 2.

Таблица 2 – Расписание проведения тестирования

ФИО	Дата	Описание	Длительность, ч
Савило Ксения Вячеславовна	12.04.2026	Разработка тест-кейсов	2
	13.04.2026	Тестирование программного обеспечения	2
	13.04.2026	Анализ выполнения тест-кейсов	1
	17.04.2026	Тестирование программного обеспечения. Проведение регрессионного тестирования	2
	17.04.2026	Составление отчёта о результатах тестирования	1

Элементы программы были проверены, и было установлено, что все они работают правильно и выполняют задачи, указанные в процедурах.

Статистика по всем дефектам представлена в таблице 3.

Таблица 3 – Статистика по всем дефектам

Статус	Количество	Важность			
		Низкая	Средняя	Высокая	Критическая
Найдено	3	3	0	0	0
Исправлено	3	3	0	0	0
Проверено	0	0	0	0	0
Открыто заново	0	0	0	0	0
Отклонено	0	0	0	0	0

Таким образом, после проведения тестирования, критических ошибок выявлено не было, найденные ошибки были исправлены. Опираясь на вышеперечисленное, можно сделать вывод о стабильности работы программного продукта и его основного функционала.

8 Применение

Данный ПП — это компьютерная игра, сочетающая в себе элементы симулятора карьеры (моделирование пути выпускника) и приключенческой игры с обучающими головоломками (мини-игры на знание документов и процессов колледжа).

Игра предполагает нелинейный сюжет, разделённый на ключевые главы. Игрок начинает как студент, которому необходимо освоить основы учебного процесса через серию обучающих мини-игр (сборка схем практик, заполнение дневника и так далее).

Задача игрока - трудоустроиться.

Смысл данного игрового приложения: создание многофункционального 2D-симулятора карьеры, который проводит игрока через полный цикл профессионального становления: от студента Гродненского государственного политехнического колледжа до успешно трудоустроенного специалиста.

Для того, чтобы установить программу запустить файл Setup.exe. Появится мастер установки игрового приложения «Career Start», представленный на рисунке 1.

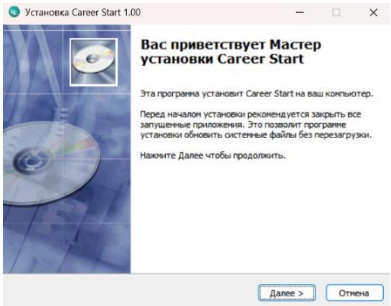

Рисунок 1 – Мастер установки

После нажатия кнопки «Далее» появляется возможность выбора места для установки программного продукта, показанная на рисунке 2.

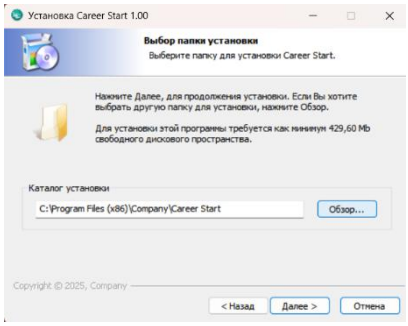

Рисунок 2 – Выбор папки установки

После нажатия на кнопку «Далее» появляется возможность выбрать места для ярлыка игрового приложения, отображенная на рисунке 3.

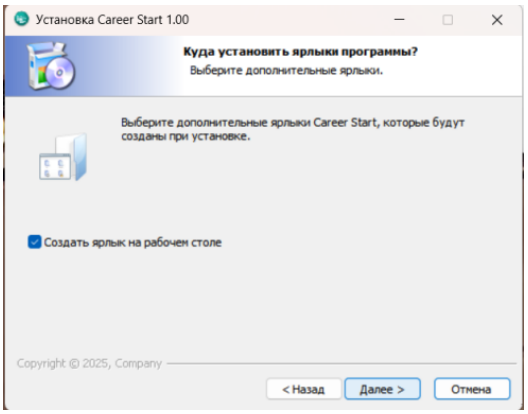

Рисунок 3 – Создание ярлыка

После нажатия на кнопку «Далее» открывается форма подтверждения установки, которая представлена на рисунке 4.

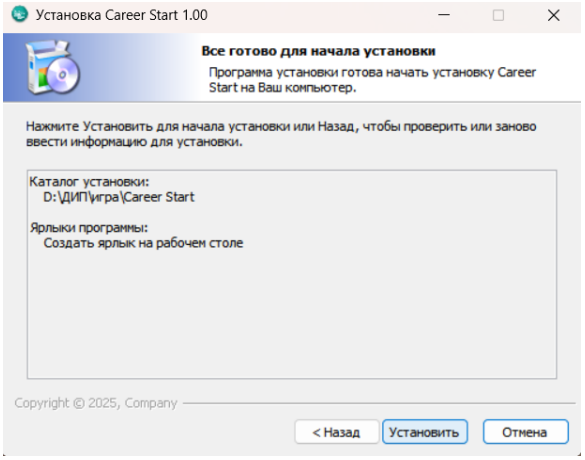

Рисунок 4 – Создание ярлыка

После успешной установки на рабочем столе появится ярлык для запуска приложения, представленный на рисунке 5.

Рисунок 5 – Ярлык приложения

После запуска приложения открывается сцена «Загрузка», представленная на рисунке 6.

Рисунок 6 – Загрузка приложения

После загрузки открывается главное меню приложения, представленное на рисунке 7.

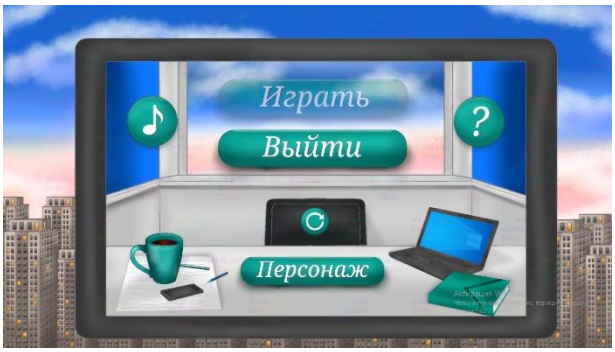

Рисунок 7 – Главное меню

У пользователя есть возможность нажать на кнопки «Выйти из игры», «Музыка», «Играть», «Справка», «Сброс прогресса», «Персонаж».

При нажатии на кнопку в виде изображения круга с вопросительным знаком будет открыта справка.

При нажатии на кнопку «Выход из игры» приложение будет закрыто.

При нажатии на кнопку в виде круга со значком перезапуска прогресс будет сброшен.

При нажатии на кнопку в виде круга со значком перезапуска прогресс будет сброшен.

При нажатии на кнопку «Музыка», изображена в виде круга со знаком ноты внутри, пользователь попадаете на сцену «Музыка», которая представлена на рисунке 8.

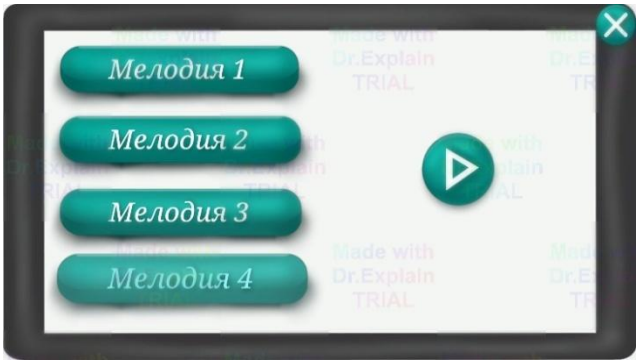

Рисунок 8 – Сцена «Музыка»

Выбрать мелодию можно кликнув по соответствующей надписи и нажатии на кнопку в виде треугольника, расположенную чуть правее.

Если музыка уже воспроизводится. то остановить её можно также нажав на кнопку в виде треугольника.

При нажатии на кнопку «Выход» пользователь вернётся на главную форму.

При нажатии на кнопку «Персонаж», расположенную на главной, пользователь попадёт на сцену настройки персонажа, которая представлена на рисунке 9.

Рисунок 9 – Сцена «Персонаж»

Не произведя настройку персонажа начать игру, будет невозможно.

Наведя на одного из двух персонажей и кликнув по нему, будет произведён выбор, который нельзя изменить, это представлено на рисунке 10

Рисунок 10 – Выбор персонажа

Далее игроку необходимо заполнить форму информации о персонаже, которая представлена на рисунке 11. Открыть эту форму можно нажав кнопку, изображённую в виде круга с троеточием.

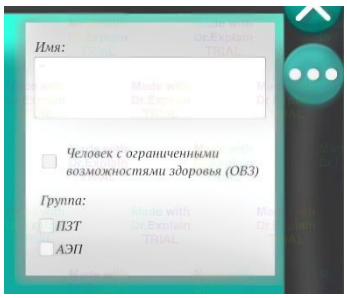

Рисунок 11 – Форма информации о персонаже

В ходе игры можно изменить имя, но пункты «ОВЗ» и «Группа» можно выбрать только единожды.

Закрыть форму можно повторным нажатием по той же кнопке.

Для возврата на главную необходимо нажать на кнопку, изображённую в виде круга с крестом.

При нажатии на кнопку «Играть», расположенную на главной, игрок попадает на сцену комнаты персонажа, которая представлена на рисунке 12.

Рисунок 12 – Сцена «Комната»

Управление персонажем осуществляется при помощи стрелок клавиатуры, или же клавиш AD: левая стрелка или клавиша A и правая стрелка или клавиша D это движения в лево и право соответственно.

На этой сцене отображается количество заработанных звёзд в левом верхнем углу на рисунке звезды.

При нажатии на кнопку со знаком дома произойдёт переход на главную сцену.

При нажатии на кнопку, изображённую в виде телефона, произойдёт переход на сцену «Телефон», которая представлена на рисунке 13

Рисунок 13 – Сцена «Телефон»

Нажав на кнопку, изображённую в виде круга с крестом происходит переход на предыдущую сцену.

Нажав на кнопку «Документы» игрок увидит перечень документов, как представлено на рисунке 14.

Открыть документ можно нажав на него.

Чтобы вернуться на главный экран телефона нужно нажать на стрелку в левом верхнем углу.

Рисунок 14 – Перечень документов

Нажав на кнопку с тремя вопросительными знаками, расположенную на сцене «Комната» будет запущено диалоговое окно с описание следующего игрового шага, которое представлено на рисунке 15.

Рисунок 15 – Диалоговое окно

При нажатии на кнопку «Далее» будет воспроизведена следующая реплика. Также после запуска диалога будет открыт доступ к заданию.

Перейти к выполнению задания можно нажав на появившуюся кнопку, изображённую в виде белого прямоугольника с красным восклицательным знаком, которая изображена на рисунке 16.

Рисунок 16 – Кнопка перехода к заданию

Перемещать блоки на сцене первого задания можно наведя курсор на блок и, зажав левую кнопку мыши, перетягивать его. Слот автоматически принимает блок при соприкосновении, что показано на рисунке 17.

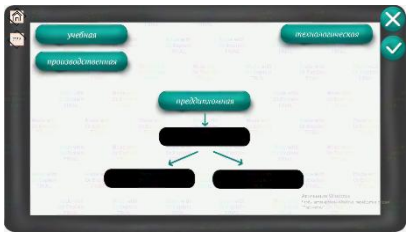

Рисунок 17 – Задание 1

Нажав на кнопку, изображённую в виде круга с птичкой, будет сдано задание.

Далее будет отображён результат и награда.

Нажав на кнопку «Далее» будет произведён переход сначала к получению фрагмента диплома, а после к следующему заданию, представленному на рисунке 18.

Рисунок 18 – Задание 2

Чтобы ответить на вопрос нужно нажать на одну из радиокнопок. Ответ может быть произведён только 1 раз.

Заключение

В ходе разработки проекта были успешно выполнены поставленные задачи по созданию игрового приложения под названием «Career Start». Целью разработки было создание интерактивной образовательно-профориентационной 2D-игры, моделирующей процесс подготовки и трудоустройства выпускника Гродненского государственного политехнического колледжа.

Основная часть проекта была реализована с использованием среды разработки Unity и языка программирования C#. Для достижения высокого качества игрового процесса и визуальной составляющей использовались такие инструменты, как Visual Studio и ibisPaint X. Это позволило добиться детализированной графики, плавной механики игры и увлекательного геймплея.

В процессе разработки были созданы UML-диаграммы, такие как диаграмма вариантов использования, диаграмма деятельности и диаграмма классов, что обеспечило чёткую структуризацию и прозрачность всех этапов работы над проектом. Это позволило эффективно организовать процесс создания игры и обеспечить достижение всех поставленных целей.

Проект был выполнен в полном соответствии с ожиданиями, а игра обладает потенциалом для дальнейшего расширения. Она может быть доработана, например, за добавления различных мини-заданий, сюжетных веток, персонажей, инвентаря, визуальных эффектов и этапов.

«Career Start» моделирует процесс подготовки и трудоустройства выпускника Гродненского государственного политехнического колледжа.

Список использованных источников

1. Для программистов, не знакомых с Unity [Электронный ресурс]. – Режим доступа: https://unity.com/ru/how-to/programming-unity/ – Дата доступа: 02.04.2026.
2. Unity с нуля: Занятие

2. C# и ООП [Электронный ресурс]. – Режим доступа: https://aelit.by/unity-s-nulya-zanyatie-2-c-i-oop/D.html – Дата доступа: 10.04.2026.

3. Полное руководство по языку программирования C# 13 и платформе .NET 9 [Электронный ресурс]. – Режим доступа: https://metanit.com/sharp/tutorial/ – Дата доступа: 12.04.2026.

4. Создаем 2D-игру на Unity: инструкция для новичка [Электронный ресурс]. – Режим доступа: https://proglib.io/p/sozdaem-2d-igru-na-unity-instrukciya dlya-novichka-2020-09-01 – Дата доступа: 18.04.2026.

5. Руководство по C# [Электронный ресурс]. – Режим доступа: https://vscode.ru/prog-lessons/sozdanie-redaktirovanie-i-udalenie-dannyih-xml-fayla-c sharp.html – Дата доступа: 22.05.2026.

Приложение Б

Диаграммы UML

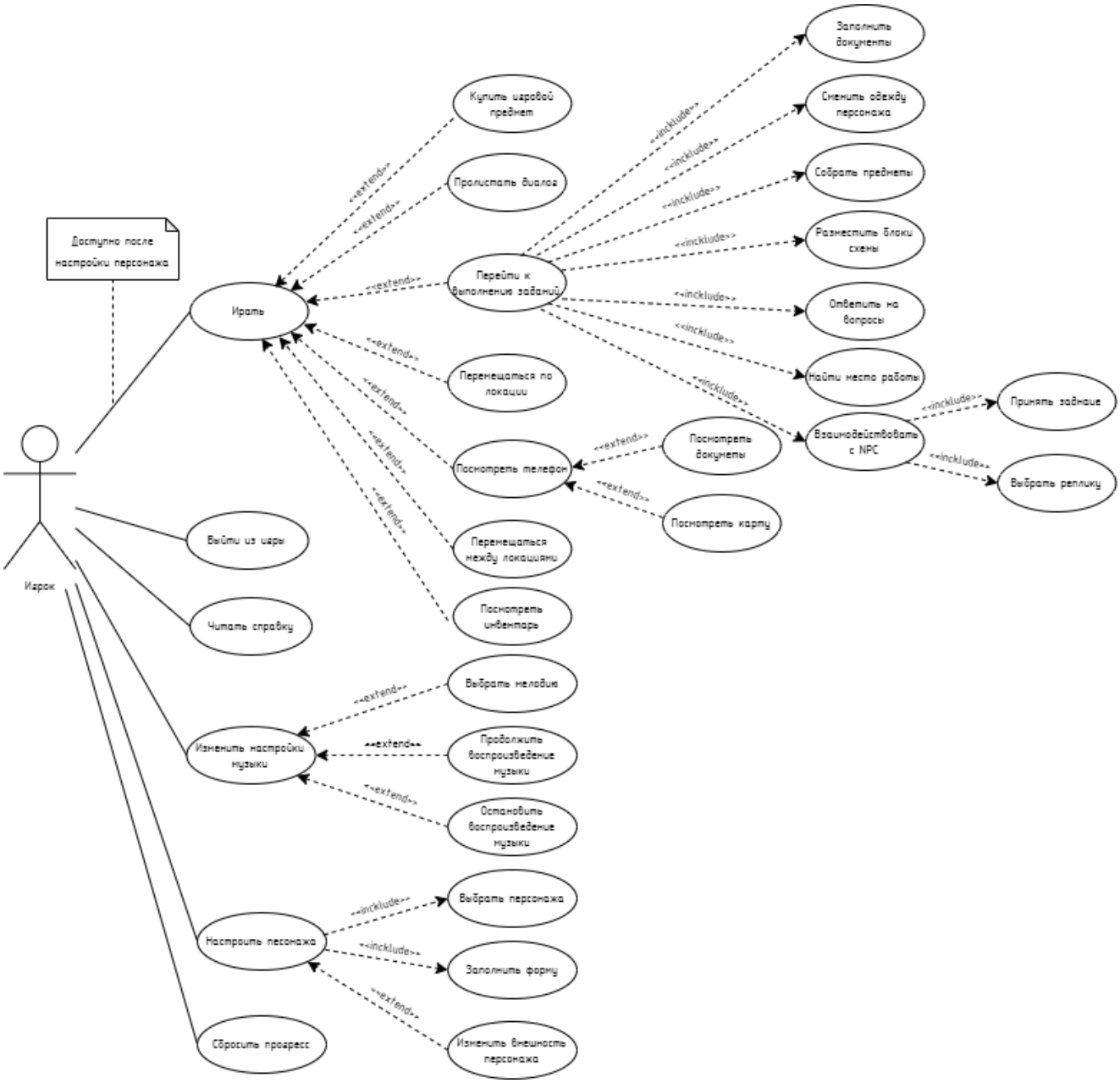

Рисунок Б.1 – Диаграмма вариантов использования

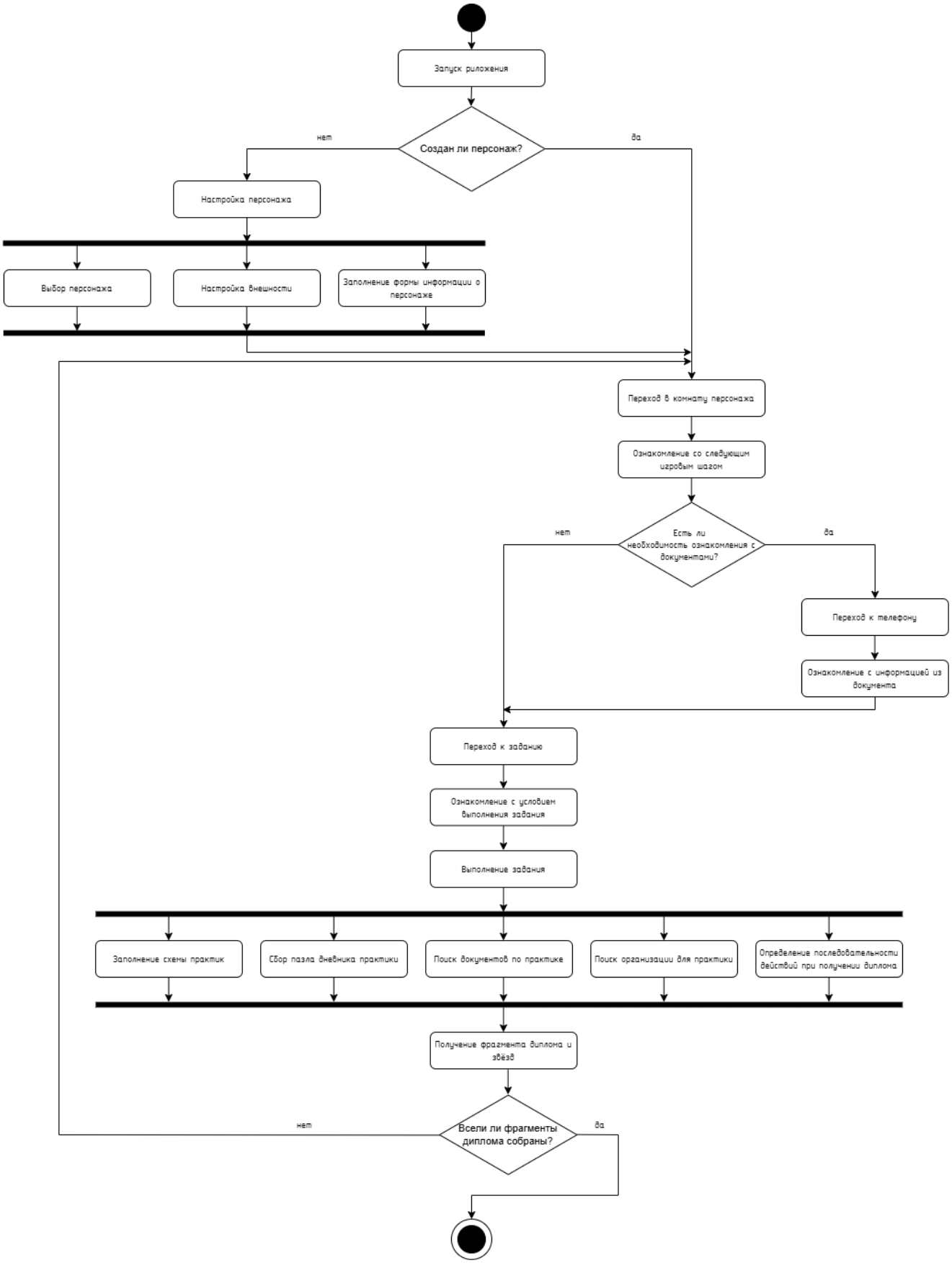

Рисунок Б.2 – Диаграмма деятельности

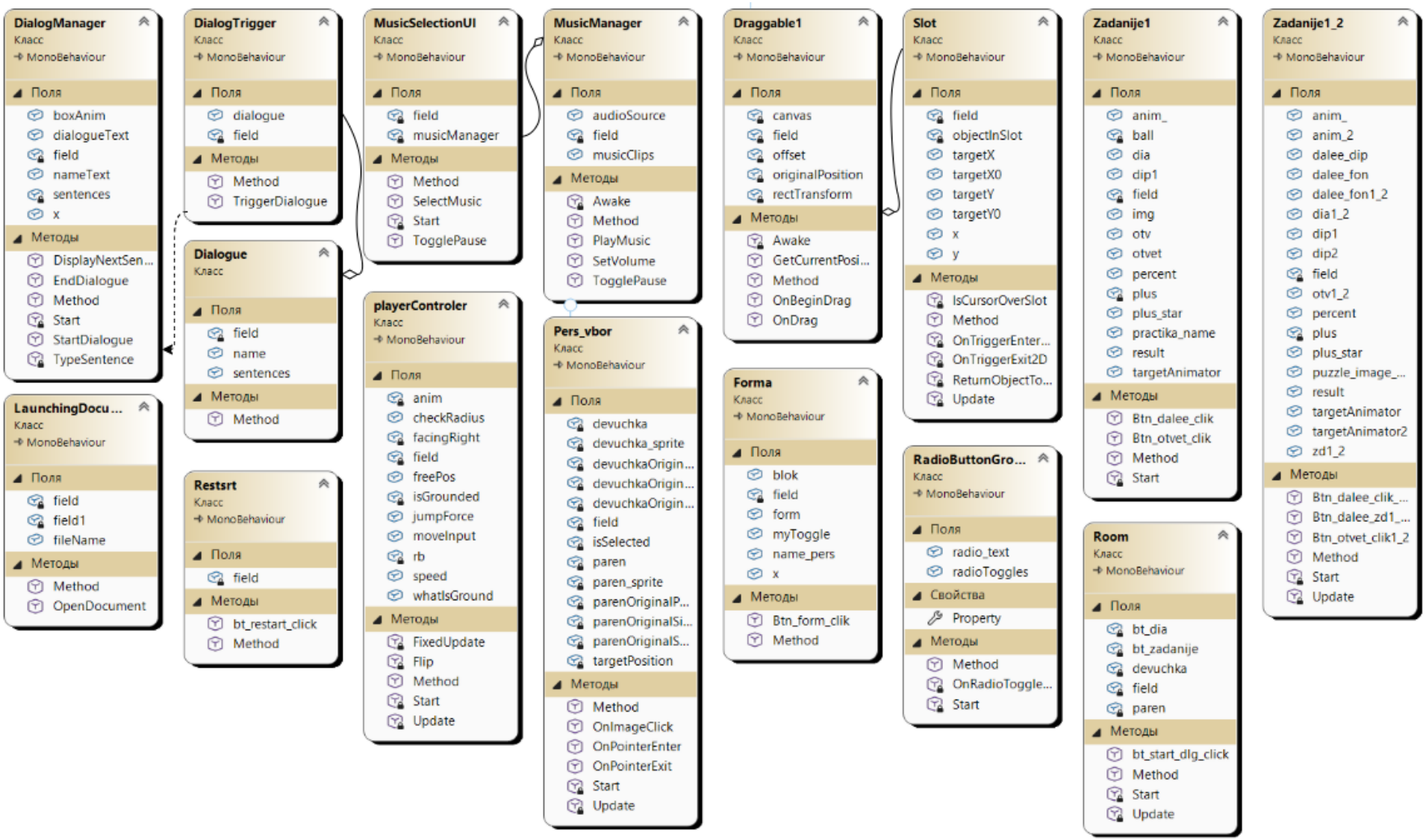

Рисунок В.3 – Диаграмма классов

Приложение В

Тест-кейсы

Таблица В.1 – Тестирование

Идентификатор	Приоритет	Заглавие и шаги выполнения	Ожидаемый результат	Фактический результат	Статус
1	2	3	4	5	6
Т_01	крайне высокий	1. Запустить игровое приложение «Career start» 2. Нажать на кнопку «Справка». 3. Нажать на раздел «Общие сведения». 4. Нажать на подраздел «О программе». 5. Нажать на подраздел «Об авторе». 6. Нажать на раздел «Руководство к применению». 7. Нажать на подраздел «Играть». 8. Нажать на подраздел «Музыка». 9. Нажать на подраздел «Персонаж». 10. Нажать на кнопку «Закрыть». 11. Нажать на кнопку «Музыка». 12. Нажать на кнопку «Мелодия 2». 13. Нажать на кнопку «Стоп». 14. Нажать на кнопку «Стоп». 15. На сцене «Музыка» нажать на кнопку «Выход». 16. Нажать на кнопку персонаж. 17. Навести курсор на женского персонажа, затем на мужского персонажа. 18. Нажать на женского персонажа. 19. Нажать на кнопку «Форма». 20. Ввести в поле «Имя». значение «Аля». 21. Нажать на чекбокс «ОВЗ». 22. Нажать на радиокнопку	1. Игра запускается. Отображается экран загрузки. Загружается главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка играть неактивна. 2. Загружается окно справки с начальной страницей и разделами: «Общие сведения» с подразделами «О программе», «Об авторе», раздел «Руководство к применению» с подразделами «Играть» и «Музыка», «Персонаж». 3. Осуществляется переход на страницу «Общие сведения» с базовой информацией. 4. Осуществляется переход на страницу «О программе» с основной информацией о приложении. 5. Осуществляется переход на страницу «Об авторе» с основной информацией о создателях приложения. 6. Осуществляется переход на страницу «Руководство к применению» с информацией по главному экрану. 7. Осуществляется переход на страницу «Играть» с основной информацией о механике игры. 8. Осуществляется переход на страницу «Музыка» с основной информацией о настройке музыки. 9. Осуществляется переход на страницу «Персонаж» с основной информацией о настройке персонажа. 10. Закрывается справка. 11. Загружается сцена «Музыка» с кнопками «Мелодия 1», «Мелодия 2», «Мелодия 3», «Мелодия 4», «Стоп», «Выход».	1. Игра запустилась. Отобразился экран загрузки. Загрузилось главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка играть неактивна. 2. Загрузилось окно справки с начальной страницей и разделами: «Общие сведения» с подразделами «О программе», «Об авторе», раздел «Руководство к применению» с подразделами «Играть» и «Музыка», «Персонаж». 3. Осуществился переход на страницу «Общие сведения» с базовой информацией. 4. Осуществился переход на страницу «О программе» с основной информацией о приложении. 5. Осуществился переход на страницу «Об авторе» с основной информацией о создателях приложения. 6. Осуществился переход на страницу «Руководство к применению» с информацией по главному экрану. 7. Осуществился переход на страницу «Играть» с основной информацией о механике игры. 8. Осуществился переход на страницу «Музыка» с основной информацией о настройке музыки. 9. Осуществился переход на страницу «Персонаж» с основной информацией о настройке персонажа. 10. Закрылась справка. 11. Загрузилась сцена «Музыка» с кнопками «Мелодия 1», «Мелодия 2», «Мелодия 3», «Мелодия 4», «Стоп», «Выход».	Пройдено

Продолжение таблицы Б.1

1	2	3	4	5	6
		«ПЗТ» в разделе «Группа». 23. Нажать на радиокнопку «АЭП» в разделе «Группа» 24. Нажать на кнопку «Форма». 25. Нажать на кнопку «Форма». 26. Нажать на кнопку «Закрыть». 27. Нажать на кнопку «Играть» 28. Зажать клавиши по порядку: «A», «D», «W», «LeftArrow», «RightArrow», «Upward Arrow». 29. Нажать на кнопку «Телефон». 30. Нажать на кнопку «Документы». 31. Нажать на кнопку документа. 32. Закрыть документ. 33. Нажать на кнопку возврата на главную страницу телефона. 34. Нажать кнопку «Закрыть» 35. Нажать на кнопку «Диалог». 36. Нажать на кнопку «Далее». 37. Пролистать диалог до конца. 38. Нажать на кнопку «Задание». 39. Нажать на кнопку «Диалог». 40. Пролистать диалог нажав на кнопку «Далее» 41. Навести курсор на блок, зажать левую кнопку мыши и перенести блок к слоту. 42. Повторить пункт 39 в отношении других блоков и слотов. 43. Нажать на кнопку «Ответить». 44. Нажать на кнопку «Далее». 45. Нажать на кнопку «Далее». 46. Нажать на первую	12. Запускается мел2_information-corporate-advertising-music-252179. 13. Прекращается воспроизведение мел2_information-corporate-advertising-music-252179. 14. Продолжается воспроизведение мел2_information-corporate-advertising-music-252179. 15. Загружается главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка играть неактивна. 16. Загружается сцена «Персонаж» с двумя персонажами, кнопкой «Форма» и «Выход» 17. Изображение женского персонажа увеличивается и подсвечивается, изображение мужского персонажа увеличивается и подсвечивается. 18. Мужской персонаж исчезает, женский персонаж располагается по середине сцены. 19. Отображается форма для введения данных персонажа с полем «Имя», не отмеченным чекбоксом «ОВЗ» и не отмеченными радиокнопками «ПЗТ» и «АЭП» раздела «Группа» 20. В поле «Имя» отображается значение «Аля». 21. Чекбокс отмечается. 22. Радиокнопка «ПЗТ» отмечается, радиокнопка «АЭП» не отмечается. 23. Радиокнопка «АЭП» отмечается, радиокнопка «ПЗТ» не отмечается. 24. Форма закрывается. 25. Отображается форма для введения данных персонажа с полем «Имя», отмеченным чекбоксом «ОВЗ» и не отмеченной радиокнопкой «ПЗТ», и	12. Запускается мел2_information-corporate-advertising-music-252179. 13. прекратилось воспроизведение мел2_information-corporate-advertising-music-252179. 14. Продолжилось воспроизведение мел2_information-corporate-advertising-music-252179. 15. Загрузилось главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка играть неактивна. 16. Загрузилась сцена «Персонаж» с двумя персонажами, кнопкой «Форма» и «Выход» 17. Изображение женского персонажа увеличилось и подсветилось, изображение мужского персонажа увеличилось и подсветилось. 18. Мужской персонаж исчез, женский персонаж расположился по середине сцены. 19. Отразилась форма для введения данных персонажа с полем «Имя», не отмеченным чекбоксом «ОВЗ» и не отмеченными радиокнопками «ПЗТ» и «АЭП» раздела «Группа» 20. В поле «Имя» отобразилось значение «Аля». 21. Чекбокс отметился. 22. Радиокнопка «ПЗТ» отметилась, радиокнопка «АЭП» не отметилась. 23. Радиокнопка «АЭП» отметилась, радиокнопка «ПЗТ» не отметилась. 24. Форма закрылась. 25. Отобразилась форма для введения данных персонажа с полем «Имя», отмеченным чекбоксом «ОВЗ» и не отмеченной радиокнопкой «ПЗТ», и	

Продолжение таблицы Б.1

1	2	3	4	5	6
		радиокнопку варианта ответа на первый вопрос. 47. Нажать на первую кнопку последнего вопроса. 48. Нажать на кнопку «Ответить». 49. Нажать на кнопку «Далее». 50. Нажать на кнопку «Закрыть». 51. Нажать на кнопку указателя третьего задания. 52. Нажать на кнопку «Главная». 53. Нажать на кнопку «Сбросить прогресс». 54. Нажать на кнопку «Персонаж». 55. Навести курсор на мужского персонажа. 56. Нажать на мужского персонажа. 57. Нажать на кнопку «Закрыть». 58. Нажать на кнопку «Персонаж». 59. Нажать на кнопку «Форма». 60 Ввести в поле «Имя» значение «Ник». 61. Нажать на радиокнопку «ПЗТ» в разделе «Группа». 62. Нажать кнопку «Закрыть». 63. Нажать на кнопку «Играть». 64. Нажать на кнопку «Главная». 65. Нажать на кнопку «Выйти».	отмеченной радиокнопкой «АЭП» раздела «Группа». Возможность изменения данных за исключением имени блокируется. 26. Загружается главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж» и «Выход из игры». Кнопка играть активна. 27. Запускается сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 28. Персонаж идёт влево, персонаж идёт вправо, персонаж подпрыгивает, персонаж идёт влево, персонаж идёт вправо, персонаж подпрыгивае. 29. Запускается сцена «Телефон» с кнопками «Главная», «Выход», «Документы». 30. Отображается перечень доступных документов и кнопка возврата на главную страницу телефона. 31. Документ запускается в приложении по умолчанию. 32. Документ закрывается. 33. Отображается главная страница телефона. 34. Запускается сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 35. Отображается диалоговое окно с репликой и кнопкой «Далее». Отображается кнопка «Задание». 36. Реплика диалога сменяется. 37. Диалоговое окно исчезает. 38. Загружается сцена «Задание» с первым заданием, с кнопками «Главная», «Выход», «Диалог», блоками схемы и ячейками, кнопкой «Ответить». 39. Отображается диалоговое окно.	отмеченной радиокнопкой «АЭП» раздела «Группа». Возможность изменения данных за исключением имени блокировалась. 26. Загрузилось главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж» и «Выход из игры». Кнопка играть активна. 27. Запустилась сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 28. Персонаж пошёл влево, персонаж пошёл вправо, персонаж подпрыгнул, персонаж пошёл влево, персонаж пошёл вправо, персонаж подпрыгнул. 29. Запустилась сцена «Телефон» с кнопками «Главная», «Выход», «Документы». 30. Отобразился перечень доступных документов и кнопка возврата на главную страницу телефона. 31. Документ запустился в приложении по умолчанию. 32. Документ закрылся. 33. Отобразилась главная страница телефона. 34. Запустилась сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 35. Отобразилось диалоговое окно с репликой и кнопкой «Далее». Отобразилась кнопка «Задание». 36. Реплика диалога сменилась. 37. Диалоговое окно исчезло. 38. Загрузилась сцена «Задание» с первым заданием, с кнопками «Главная», «Выход», «Диалог», блоками схемы и ячейками, кнопкой «Ответить». 39. отобразилось диалоговое окно.	

Продолжение таблицы Б.1

1	2	3	4	5	6
			40. Реплики сменяются, диалоговое окно исчезает. 41. Блок перемещается, слот принимает блок. 42. Схема заполняется. 43. Отображается изображение с указание процента правильности выполнения и количеством полученных звёзд, а также кнопка «Далее». Кнопка «Диалог» скрывается. 44. Отображается фрагмент диплома с количеством пятен пропорциональным проценту выполнения. 45. Отображается второе задание. С изображением пазла, вопросами и вариантами ответа в виде радиокнопок. 46. Отображается соответствующий элемент пазла. 47. Соответствующий элемент пазла не появляется. 48. Отображается изображение с указание процента правильности выполнения и количеством полученных звёзд, а также кнопка «Далее». Кнопка «Диалог» скрывается. 49. Отображается фрагмент диплома с количеством пятен пропорциональным проценту выполнения. 50. Запускается сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 51 Запускается сцена «Вход в колледж» с кнопками: «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 52. Загружается главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры».	40. Реплики сменились, диалоговое окно исчезло. 41. Блок переместился, слот принял блок. 42. Схема заполняется. 43. отобразилось изображение с указание процента правильности выполнения и количеством полученных звёзд, а также кнопка «Далее». Кнопка «Диалог» скрылась. 44. Отобразился фрагмент диплома с количеством пятен пропорциональным проценту выполнения. 45. Отобразилось второе задание. С изображением пазла, вопросами и вариантами ответа в виде радиокнопок. 46. Отобразился соответствующий элемент пазла. 47. Отобразился элемент пазла не появляется. 48. Отобразилось изображение с указание процента правильности выполнения и количеством полученных звёзд, а также кнопка «Далее». Кнопка «Диалог» скрывается. 49. Отобразился фрагмент диплома с количеством пятен пропорциональным проценту выполнения. 50. Запустилась сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 51. запустилась сцена «Вход в колледж» с кнопками: «Комната» с кнопками «Главная», «Телефон», «Диалог», женским персонажем и указанным текущим количеством звёзд. 52. Загрузилось главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры».	

Продолжение таблицы Б.1

1	2	3	4	5	6
			53. Игровой прогресс сбрасывается, кнопка «Играть» неактива. 54. Загружается сцена «Персонаж» с двумя персонажами, кнопкой «Форма» и «Выход» 55. Изображение мужского персонажа увеличивается и подсвечивается. 56. Женский персонаж исчезает, мужской персонаж располагается по середине сцены. 57. Загружается главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка «Играть» неактивна. 58. Загружается сцена «Персонаж» с мужским персонажем, кнопкой «Форма» и «Выход». 59. Отображается форма для введения данных персонажа с полем «Имя», не отмеченным чекбоксом «ОВЗ» и не отмеченными радиокнопками «ПЗТ» и «АЭП» раздела «Группа» 60. В поле «Имя» отображается значение «Ник». 61. Отмечается радиокнопка «ПЗТ» в разделе группа. 62. Загружается главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка «Играть» активна. 63. Запускается сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», мужским персонажем и указанным текущим количеством звёзд. 64. Загружается главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». 65. Приложение закрывается.	53. Игровой прогресс сбросился, кнопка «Играть» неактива. 54. Загрузилась сцена «Персонаж» с двумя персонажами, кнопкой «Форма» и «Выход» 55. Изображение мужского персонажа увеличилось и подсветилось. 55. Женский персонаж исчез, мужской персонаж расположился по середине сцены. 57. Загрузилось главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка «Играть» неактивна. 58. Загрузилась сцена «Персонаж» с мужским персонажем, кнопкой «Форма» и «Выход». 59. Отобразилась форма для введения данных персонажа с полем «Имя», не отмеченным чекбоксом «ОВЗ» и не отмеченными радиокнопками «ПЗТ» и «АЭП» раздела «Группа» 60. В поле «Имя» отобразилось значение «Ник». 61. Отметилась радиокнопка «ПЗТ» в разделе группа. 62. Загрузилось главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». Кнопка «Играть» активна. 63. Запустилась сцена «Комната» с кнопками «Главная», «Телефон», «Диалог», мужским персонажем и указанным текущим количеством звёзд. 64. Загрузилось главное меню игры с кнопками: «Справка», «Играть», «Музыка», «Персонаж», «Сбросить прогресс» и «Выход из игры». 65. Приложение закрылось.	

Приложение А

Листинг программы

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using System.IO;
public class DialogManager : MonoBehaviour
{
    public Text dialogueText;
    public Text nameText;
    public Animator boxAnim;
    public int x;
    private Queue<string> sentences;
    private int field;
    private void Start()
    {
        sentences = new Queue<string>();
    }
    public void StartDialogue(Dialogue dialogue)
    {
        boxAnim.SetBool("boxOpen", true);

        nameText.text = dialogue.name;
        sentences.Clear();
        foreach (string sentence in dialogue.sentences)
        {
            sentences.Enqueue(sentence);
        }
        DisplayNextSentence();
    }
    public void DisplayNextSentence()
    {
        if (sentences.Count == 0)
        {
            EndDialogue();
            return;}
        string sentence = sentences.Dequeue();
        string[] stroka = File.ReadAllLines("форма_инф_игрока.txt");
        string replica = stroka[0];
        sentence = sentence.Replace("XXXX", replica);
        StopAllCoroutines();
        StartCoroutine(TypeSentence(sentence));
    }
    IEnumerator TypeSentence(string sentence)
    {
        dialogueText.text = "";
        foreach(char letter in sentence.ToCharArray())
        {
            dialogueText.text += letter;
            yield return null;
        }
    }
    public void EndDialogue()
    {
        boxAnim.SetBool("boxOpen", false);
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using System.IO;
using UnityEngine;
using UnityEngine.UI;
public class Zadanije1_2 : MonoBehaviour
{
    public Image zd1_2;
    public Button otv1_2;
    public Button dia1_2;
    public Button dalee_fon1_2;
    public Button dalee_fon;
    public Image dip1;
    public Image dip2;
    public Button dalee_dip;
    public Animator targetAnimator2;
    public int anim_2;
    public Image result;
    public Text percent;
    public Text plus_star;
    private int plus;
    public Animator targetAnimator;
    public int anim_;
    public Image[] puzzle_image_mas;
    private int field;
    private void Start()
    {
        string[] stroka5 = File.ReadAllLines("Выполнение задания
труд.txt");
        if ((stroka5[0] == "1") && (stroka5[1] == "1"))
        {
            zd1_2.gameObject.SetActive(false);
            otv1_2.gameObject.SetActive(false);
            dia1_2.gameObject.SetActive(false);
            dip1.gameObject.SetActive(true);
            dip2.gameObject.SetActive(true);
            dalee_dip.gameObject.SetActive(false);
            string[] stroka4 = File.ReadAllLines("процент выполнения
задания.txt");
            if (int.Parse(stroka4[1]) == 100)
            { anim_2 = 1; }
            else if ((int.Parse(stroka4[1]) >= 50) && (int.Parse(stroka4[1])
< 100))
            { anim_2 = 2; }
            else
            { anim_2 = 3; }
            if (targetAnimator2 != null)
            {
                targetAnimator2.SetInteger("anim0", anim_2);
                targetAnimator2.SetTrigger("MoveTrigger");
            }
            else
            {
                Debug.LogError("Animator не назначен в инспекторе!");
            }
            if (int.Parse(stroka4[0]) == 100)
            { anim_ = 1; }
            else if ((int.Parse(stroka4[0]) >= 50) && (int.Parse(stroka4[0])
< 100))
            { anim_ = 2; }
            else
            { anim_ = 3; }
            if (targetAnimator != null)
            {
                targetAnimator.SetInteger("anim0", anim_);
                targetAnimator.SetTrigger("MoveTrigger");
            }
            else
            {
                Debug.LogError("Animator не назначен в инспекторе!");
            }}}
    void Update()
    {
        string[] stroka = File.ReadAllLines("сбор пазла.txt");
        for (int i = 0; i < stroka.Length; i++)
        {
            if (stroka[i] == "1")
            {
                puzzle_image_mas[i].gameObject.SetActive(true);
```

```
            }
        }
    }
    public void Btn_otvet_clik1_2()
    {
        zd1_2.gameObject.SetActive(false);
        string[] stroka5 = File.ReadAllLines("Выполнение задания
труд.txt");
            int ball = 0;
            string[] stroka1 = File.ReadAllLines("сбор пазла.txt");
            for (int i = 0; i < stroka1.Length; i++)
            {
                if (stroka1[i] == "1")
                {
                    ball++;
                }}
            result.gameObject.SetActive(true);
        dalee_fon.gameObject.SetActive(false);
        dalee_fon1_2.gameObject.SetActive(true);
        int perc = (ball * 100) / 9;
            percent.text = (perc).ToString() + "%";
            if ((perc >= 25) && (perc <= 50))
            {
                plus = 1;
            }
            else if ((perc >= 51) && (perc <= 75))
            {
                plus = 2;
            }
            else if ((perc >= 76) && (perc <= 99))
            {
                plus = 3;
            }
            else if ((perc == 100))
            {
                plus = 4;
            }
            else
            {
                plus = 0;
            }
            otv1_2.gameObject.SetActive(false);
            dia1_2.gameObject.SetActive(false);
            plus_star.text = plus.ToString();
            string[] stroka2 = File.ReadAllLines("Звёзды.txt");
            stroka2[0] = (int.Parse(stroka2[0]) + plus).ToString();
            StreamWriter f1 = new StreamWriter("Звёзды.txt", false);
            for (int k = 0; k < stroka2.Length; k++)
            {
                f1.WriteLine(stroka2[k]);
            }
            f1.Close();
            string[] stroka3 = File.ReadAllLines("Выполнение задания
труд.txt");
            stroka3[1] = "1";
            StreamWriter f3 = new StreamWriter("Выполнение задания
труд.txt", false);
            for (int k = 0; k < stroka3.Length; k++)
            {
                f3.WriteLine(stroka3[k]);
            }
            f3.Close();
            string[] stroka4 = File.ReadAllLines("процент выполнения
задания.txt");
            stroka4[1] = perc.ToString();
            StreamWriter f4 = new StreamWriter("процент выполнения
задания.txt", false);
            for (int k = 0; k < stroka4.Length; k++)
            {
                f4.WriteLine(stroka4[k]);
            }
            f4.Close();
    }
    public void Btn_dalee_zd1_2_clik()
    {
        dip1.gameObject.SetActive(false);
        zd1_2.gameObject.SetActive(true);
        otv1_2.gameObject.SetActive(true);
        dia1_2.gameObject.SetActive(true);
    }
    public void Btn_dalee_clik_end()
    {
        zd1_2.gameObject.SetActive(false);
        otv1_2.gameObject.SetActive(false);
        dia1_2.gameObject.SetActive(false);
        dip1.gameObject.SetActive(true);
        dip2.gameObject.SetActive(true);
        dalee_dip.gameObject.SetActive(false);
        string[] stroka4 = File.ReadAllLines("процент выполнения
задания.txt");
        if (int.Parse(stroka4[1]) == 100)
        { anim_2 = 1; }
        else if ((int.Parse(stroka4[1]) >= 50) && (int.Parse(stroka4[1]) <
100))
        { anim_2 = 2; }
        else
        { anim_2 = 3; }
        if (targetAnimator2 != null)
        {
            targetAnimator2.SetInteger("anim0", anim_2);
            targetAnimator2.SetTrigger("MoveTrigger");
        }
        else
        {
            Debug.LogError("Animator не назначен в инспекторе!");
        }
        if (int.Parse(stroka4[0]) == 100)
        { anim_ = 1; }
        else if ((int.Parse(stroka4[0]) >= 50) && (int.Parse(stroka4[0]) <
100))
        { anim_ = 2; }
        else
        { anim_ = 3; }
        if (targetAnimator != null)
        {
            targetAnimator.SetInteger("anim0", anim_);
            targetAnimator.SetTrigger("MoveTrigger");
        }
        else
        {
            Debug.LogError("Animator не назначен в инспекторе!");
        }
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using UnityEngine;
using System.IO;
using UnityEngine.UI;
public class Stars : MonoBehaviour
{
    [SerializeField] private Text kol_stars;
    void Update()
    {
        string[] stroka = File.ReadAllLines("Звёзды.txt");
```

```
        kol_stars.text = stroka[0];
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using System.IO;
public class Room : MonoBehaviour
{
    [SerializeField] private GameObject devuchka;
    [SerializeField] private Image paren;
    [SerializeField] private Button[] bt_zadanije;
    [SerializeField] private Button[] bt_dia;
    private int field;
    void Update()
    {
        string[] stroka = File.ReadAllLines("Персонаж.txt");
        if (stroka[1] == "1")
        {
            paren.gameObject.SetActive(true);
        }
        else if (stroka[0] == "1")
        {
            devuchka.gameObject.SetActive(true);
        }
        string[] stroka1 = File.ReadAllLines("Переход к заданию.txt");
        if(stroka1[0] == "1")
        {
            bt_zadanije[0].gameObject.SetActive(true);
        }
        string[] stroka5 = File.ReadAllLines("Выполнение задания
труд.txt");
        if ((stroka5[0] == "1") && (stroka5[1] == "1"))
        {
            bt_zadanije[0].gameObject.SetActive(false);
            bt_dia[0].gameObject.SetActive(false);
            bt_dia[1].gameObject.SetActive(true);
        }
        }
    public void bt_start_dlg_click()
    {
        bt_zadanije[0].gameObject.SetActive(true);
        string[] stroka5 = File.ReadAllLines("Выполнение задания
труд.txt");
        if ((stroka5[0] == "1") && (stroka5[1] == "1"))
        {
            bt_zadanije[0].gameObject.SetActive(false);}}
    public void Method()
    {
        throw new System.NotImplementedException();
    }}

using UnityEngine;
using UnityEngine.UI;
using System.IO;
public class RadioButtonGroup : MonoBehaviour
{
    public Toggle[] radioToggles;
    public Text[] radio_text;
    public int Property
    {
        get => default;
        set
        {
        }
    }
    private void Start()
    {
        if (radioToggles == null || radioToggles.Length == 0)
        {
            Debug.LogError("Массив радиокнопок не назначен!");
            return;
        }
        // Добавляем обработчик события для каждого Toggle
        for (int i = 0; i < radioToggles.Length; i++)
        {
            int index = i; // Локальная переменная для замыкания
            radioToggles[index].onValueChanged.AddListener(delegate {
OnRadioToggleValueChanged(index); });
        }
        string[] stroka = File.ReadAllLines("форма_инф_игрока.txt");

        if (stroka[2] != (-1).ToString())
        {
            for (int i = 0; i < radio_text.Length; i++)
            {
                if (stroka[2]== radio_text[i].text) {
                radioToggles[i].isOn = true;
                }}}}
    private void OnRadioToggleValueChanged(int changedIndex)
    {
        if (radioToggles[changedIndex].isOn)
        {
            for (int i = 0; i < radioToggles.Length; i++)
            {
                if (i != changedIndex)
                {
                    radioToggles[i].isOn = false;
                }
            }
            string[] stroka = File.ReadAllLines("форма_инф_игрока.txt");
            stroka[2] = radio_text[changedIndex].text;
            StreamWriter f1 = new
StreamWriter("форма_инф_игрока.txt", false);
            for (int k = 0; k < stroka.Length; k++)
            {
                f1.WriteLine(stroka[k]);
            }
            f1.Close();
        }
    }

    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using UnityEngine;
using UnityEngine.UI;
using UnityEngine.EventSystems;
using System.IO;
public class Pers_vbor : MonoBehaviour, IPointerEnterHandler,
IPointerExitHandler
{
    [Header("First Image")]
    [SerializeField] private Image devuchka;
    [SerializeField] private Sprite devuchka_sprite;
    [Header("Second Image")]
    [SerializeField] private Image paren;
    [SerializeField] private Sprite paren_sprite;
    [Header("Click Settings")]
    [SerializeField] private Vector2 targetPosition = new Vector2(0, 0);
    private Vector2 devuchkaOriginalSize, parenOriginalSize;
    private Vector2 devuchkaOriginalPosition, parenOriginalPosition;
    private Sprite devuchkaOriginalSprite, parenOriginalSprite;
```

```
    private bool isSelected = false;
    private int field;
    void Start()
    {
        if (devuchka != null)
        {
            devuchkaOriginalSize = devuchka.rectTransform.sizeDelta;
            devuchkaOriginalSprite = devuchka.sprite;
            devuchkaOriginalPosition =
devuchka.rectTransform.anchoredPosition;
        }
        if (paren != null)
        {
            parenOriginalSize = paren.rectTransform.sizeDelta;
            parenOriginalSprite = paren.sprite;
            parenOriginalPosition = paren.rectTransform.anchoredPosition;
        }
    }
    void Update()
    {
        string[] stroka = File.ReadAllLines("Персонаж.txt");
        if (int.Parse(stroka[0]) == 1)
        {
            paren.gameObject.SetActive(false);
            devuchka.rectTransform.anchoredPosition = targetPosition;
        }
        else if (int.Parse(stroka[1]) == 1)
        {
            devuchka.gameObject.SetActive(false);
            paren.rectTransform.anchoredPosition = targetPosition;
        }
    }
        public void OnImageClick(Image clickedImage)
    {
        if (isSelected) return;
        isSelected = true;
        string[] stroka = File.ReadAllLines("Персонаж.txt");
        if (clickedImage == devuchka)
        {
            devuchka.rectTransform.anchoredPosition = targetPosition;

            if (paren != null) paren.gameObject.SetActive(false);
            stroka[0] = "1";
        }
        else if (clickedImage == paren)
        {
            paren.rectTransform.anchoredPosition = targetPosition;
            if (devuchka != null) devuchka.gameObject.SetActive(false);
            stroka[1] = "1";
        }

        StreamWriter f1 = new StreamWriter("Персонаж.txt", false);
        for (int k = 0; k < stroka.Length; k++)
        {
            f1.WriteLine(stroka[k]);
        }
        f1.Close();
    }
    public void OnPointerEnter(PointerEventData eventData)
    {
        var target = eventData.pointerEnter.GetComponent<Image>();
        if (target == devuchka)
        {
            devuchka.rectTransform.sizeDelta = new Vector2(300, 720);
            if (devuchka_sprite != null) devuchka.sprite = devuchka_sprite;
        }
        else if (target == paren)
        {
            paren.rectTransform.sizeDelta = new Vector2(312, 826);
            if (paren_sprite != null) paren.sprite = paren_sprite;
        }
    }
    public void OnPointerExit(PointerEventData eventData)
    {
        var target = eventData.pointerEnter.GetComponent<Image>();

        if (target == devuchka)
        {
            devuchka.rectTransform.sizeDelta = devuchkaOriginalSize;
            devuchka.sprite = devuchkaOriginalSprite;
        }
        else if (target == paren)
        {
            paren.rectTransform.sizeDelta = parenOriginalSize;
            paren.sprite = parenOriginalSprite;
        }
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using UnityEngine;
using UnityEngine.UI;
using UnityEngine.EventSystems;
public class knopka : MonoBehaviour, IPointerEnterHandler,
IPointerExitHandler
{
    [Header("Image effect")]
    [SerializeField] private Image bt;
    [SerializeField] private Sprite bt_sprite;
    private Vector2 btOriginalSize;
    private Sprite btOriginalSprite;
    public float x;
    public float y;
    void Start()
    {
        if (bt != null)
        {
            btOriginalSize = bt.rectTransform.sizeDelta;
            btOriginalSprite = bt.sprite;

        }
    }
    public void OnPointerEnter(PointerEventData eventData)
    {
        var target = eventData.pointerEnter.GetComponent<Image>();
        if (target == bt)
        {
            bt.rectTransform.sizeDelta = new Vector2(x , y);
            if (bt_sprite != null) bt.sprite = bt_sprite;
        }
    }
    public void OnPointerExit(PointerEventData eventData)
    {
        var target = eventData.pointerEnter.GetComponent<Image>();
        if (target == bt)
        {
            bt.rectTransform.sizeDelta = btOriginalSize;
            bt.sprite = btOriginalSprite;
        }
    }
}

using UnityEngine;
using UnityEngine.UI;
using System.IO;
```

```
public class Forma : MonoBehaviour
{
    public GameObject form;
    public int x = 0;
    public InputField name_pers;
    public Toggle myToggle;
    public Image blok;
    private int field;
    public void Btn_form_clik(GameObject clickedButton)
    {
        string[] stroka = File.ReadAllLines("форма_инф_игрока.txt");
        if (x == 0)
        {
            if(clickedButton.name != "выход")
            { form.gameObject.SetActive(true);
            if (stroka[2] != "-1")
                {
                    blok.gameObject.SetActive(true);
                }
            }
            x = 1;
            name_pers.text= stroka[0];
            if(stroka[1] == "1")
            {
                myToggle.isOn = true;
            }
            else
                    {
                myToggle.isOn = false;
            }
        }
        else
        {
            form.gameObject.SetActive(false);
            blok.gameObject.SetActive(false);
            x = 0;
            stroka[0] = name_pers.text;
            if (myToggle.isOn == true)
            {
                stroka[1] = "1";
            }
            else
            {
                stroka[1] = "0";
            }
            StreamWriter f1 = new
StreamWriter("форма_инф_игрока.txt", false);
            for (int k = 0; k < stroka.Length; k++)
            {
                f1.WriteLine(stroka[k]);
            }
            f1.Close();
        }
        }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using UnityEngine;
using UnityEngine.UI;
using UnityEngine.EventSystems;
using System.IO;
using Unity.VisualScripting;
public class Blok_play : MonoBehaviour
{
    [SerializeField] private Image bt_play;
    [SerializeField] private Button bt_blok;
    [SerializeField] private Sprite bt_sprite_no_active;
    void Update()
    {
        string[] stroka = File.ReadAllLines("Персонаж.txt");
        string[] stroka1 = File.ReadAllLines("форма_инф_игрока.txt");
        if (((stroka[0]=="0")&&(stroka[1] == "0")) || (stroka1[2]=="-1"))
        {
            bt_play.sprite = bt_sprite_no_active;
            bt_blok.gameObject.SetActive(true);
        }
    }
}

using System.IO;
using UnityEngine;
public class Slot : MonoBehaviour
{
    [Header("Позиция куда переместится объект")]
    public float targetX;
    public float targetY;
    [Header("Позиция возврата при нажатии Backspace")]
    public float[] targetX0;
    public float[] targetY0;
    [Header("Дополнительные параметры")]
    public string y;
    public int x;
    private GameObject objectInSlot;
    private int field;

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (objectInSlot == null)
        {
            Debug.Log($"Объект {collision.name} коснулся слота
{name}");
            Draggable1 draggable =
collision.GetComponent<Draggable1>();
            if (draggable != null)
            {
                objectInSlot = collision.gameObject;
                Destroy(draggable);
                Debug.Log($"Объект обездвижен");
                RectTransform draggableRect =
collision.GetComponent<RectTransform>();
                draggableRect.anchoredPosition = new Vector2(targetX,
targetY);
                Debug.Log($"Объект {collision.name} перемещен в
позицию ({targetX}, {targetY})");
                string[] stroka = File.ReadAllLines("Задание1(вариант
сбора).txt");
                if ((y=="Slot1")&& (collision.name== "поле 1"))
                {
                    stroka[0] = "1";
                }
                if  ((y == "Slot2") && (collision.name == "поле 2"))
                {
                    stroka[1] = "1";
                }
                if((y == "Slot3")  && ((collision.name == "поле 3") ||
(collision.name == "поле 4")))
                {
                    stroka[2] = "1";
                }
                if ((y == "Slot4") && ((collision.name == "поле 3") ||
(collision.name == "поле 4")))
                {
                    stroka[3] = "1";
                }
```

```
            StreamWriter f = new StreamWriter("Задание1(вариант
сбора).txt", false);
            for (int k = 0; k < stroka.Length; k++)
            {
                f.WriteLine(stroka[k]);
            }
            f.Close();
        }
    }}

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Backspace))
        {
            if (objectInSlot != null && IsCursorOverSlot())
            {
                ReturnObjectToStartPosition();
            }
        }
    }
    private bool IsCursorOverSlot()
    {
        RectTransform slotRect = GetComponent<RectTransform>();
        if (slotRect == null) return false;


        Vector2 localCursor;
        bool cursorOverSlot =
RectTransformUtility.ScreenPointToLocalPointInRectangle(
            slotRect,
            Input.mousePosition,
            null,
            out localCursor
        );
        if (cursorOverSlot)
        {
            Rect rect = slotRect.rect;
            bool isInside = rect.Contains(localCursor);
            if (isInside)
            {
                Debug.Log($"Курсор над слотом {name} в позиции
{localCursor}");
            }

            return isInside;
        }
        return false;
    }
    private void ReturnObjectToStartPosition()
    {
        if (objectInSlot != null)
        {
            Debug.Log($"Возвращаем объект {objectInSlot.name} в
начальную позицию");
            RectTransform objRect =
objectInSlot.GetComponent<RectTransform>();
            if (objectInSlot.name == "поле 1")
            {
                objRect.anchoredPosition = new Vector2(targetX0[0],
targetY0[0]);
            }
            else if (objectInSlot.name == "поле 2")
            {
                objRect.anchoredPosition = new Vector2(targetX0[1],
targetY0[1]);
            }
            else if (objectInSlot.name == "поле 3")
            {
                objRect.anchoredPosition = new Vector2(targetX0[2],
targetY0[2]);
            }
            else if (objectInSlot.name == "поле 4")
            {
                objRect.anchoredPosition = new Vector2(targetX0[3],
targetY0[3]);
            }
            objectInSlot.AddComponent<Draggable1>();
            Debug.Log($"Объект {objectInSlot.name} возвращен");
            objectInSlot = null;
        }
    }

    private void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.gameObject == objectInSlot)
        {
            Debug.Log($"Объект {objectInSlot.name} вышел из слота");
            objectInSlot = null;
            string[] stroka = File.ReadAllLines("Задание1(вариант
сбора).txt");
            if ((y == "Slot1") && (collision.name == "поле 1"))
            {
                stroka[0] = "0";
            }
            if ((y == "Slot2") && (collision.name == "поле 2"))
            {
                stroka[1] = "0";
            }
            if ((y == "Slot3") && ((collision.name == "поле 3") ||
(collision.name == "поле 4")))
            {
                stroka[2] = "0";
            }
            if ((y == "Slot4") && ((collision.name == "поле 3") ||
(collision.name == "поле 4")))
            {
                stroka[3] = "0";
            }
            StreamWriter f = new StreamWriter("Задание1(вариант
сбора).txt", false);
            for (int k = 0; k < stroka.Length; k++)
            {
                f.WriteLine(stroka[k]);
            }
            f.Close();
        }
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System.IO;
using UnityEngine.EventSystems;
public class Draggable1 : MonoBehaviour, IDragHandler,
IBeginDragHandler
{
    private Canvas canvas;
    private RectTransform rectTransform;
    private Vector3 originalPosition;
    private Vector2 offset;
    private int field;
    void Awake()
```

```
    {
        rectTransform = GetComponent<RectTransform>();
        canvas = GetComponentInParent<Canvas>();
        originalPosition = rectTransform.anchoredPosition;
    }
    public void OnBeginDrag(PointerEventData eventData)
    {
        RectTransform rectTransform =
GetComponent<RectTransform>();
        originalPosition = rectTransform.anchoredPosition;
        offset = eventData.position -
RectTransformUtility.WorldToScreenPoint(eventData.pressEventCam
era, rectTransform.position);
    }
    public void OnDrag(PointerEventData eventData)
    {
        Vector2 position;
RectTransformUtility.ScreenPointToLocalPointInRectangle(canvas.tra
nsform as RectTransform, eventData.position, canvas.worldCamera,
out position);
        Vector2 newPosition = eventData.position - offset;
        RectTransform rectTransform =
GetComponent<RectTransform>();
        rectTransform.position = newPosition;
    }
    public Vector3 GetCurrentPosition()
    {
        return rectTransform.anchoredPosition;
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[System.Serializable]
public class Dialogue
{
    public string name;
    [TextArea(3, 10)]
    public string[] sentences;
    private int field;
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class DialogTrigger : MonoBehaviour
{
    public Dialogue dialogue;
    private int field;

    public void TriggerDialogue()
    {
        FindObjectOfType<DialogManager>().StartDialogue(dialogue);
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using System.IO;
public class DialogManager : MonoBehaviour
{
    public Text dialogueText;
    public Text nameText;
    public Animator boxAnim;
    public int x;
    private Queue<string> sentences;
    private int field;
    private void Start()
    {
        sentences = new Queue<string>();
    }
    public void StartDialogue(Dialogue dialogue)
    {
        boxAnim.SetBool("boxOpen", true);
        nameText.text = dialogue.name;
        sentences.Clear();
        foreach (string sentence in dialogue.sentences)
        {
            sentences.Enqueue(sentence);
        }
        DisplayNextSentence();
    }
    public void DisplayNextSentence()
    {
        if (sentences.Count == 0)
        {
            EndDialogue();
            return;
        }
        string sentence = sentences.Dequeue();
        string[] stroka = File.ReadAllLines("форма_инф_игрока.txt");
        string replica = stroka[0];
        sentence = sentence.Replace("XXXX", replica);
        StopAllCoroutines();
        StartCoroutine(TypeSentence(sentence));
    }
    IEnumerator TypeSentence(string sentence)
    {
        dialogueText.text = "";
        foreach(char letter in sentence.ToCharArray())
        {
            dialogueText.text += letter;
            yield return null;
        }
    }
    public void EndDialogue()
    {
        boxAnim.SetBool("boxOpen", false);
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
public class Переход : MonoBehaviour
{
    private Animator anim;
    public int levelToLoad;
```

```
    private void Start()
    {
        anim = GetComponent<Animator>();
    }
    public void FadeToLevel()
    {
        anim.SetTrigger("fade");
    }
    public void OnFadeComplete()
    {
        SceneManager.LoadScene(levelToLoad);
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
public class zagruska : MonoBehaviour
{
    private Animator anim;
    private int field;
    void Start()
    {
       StartCoroutine(gar1());
    }
    private IEnumerator gar1()
    {
        yield return new WaitForSeconds(6.5f);
        anim = GetComponent<Animator>();
        SceneManager.LoadScene(1);
        anim.SetTrigger("fade");
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using System.Collections;
using System.Collections.Generic;
using System.Diagnostics;
using UnityEngine;
public class Spravka : MonoBehaviour
{
    public string chmFilePath;
    public void OpenCHM()
    {
        Process.Start(chmFilePath);
    }
}
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System.IO;
public class Restsrt : MonoBehaviour
{
    private int field;
    public void bt_restart_click()
    {
        string[] stroka = File.ReadAllLines("Выполнение задания
труд.txt");
        StreamWriter f = new StreamWriter("Выполнение задания
труд.txt", false);
        for (int k = 0; k < stroka.Length; k++)
        {
            stroka[k] = "0";
            f.WriteLine(stroka[k]);
        }
        f.Close();

        string[] stroka1 = File.ReadAllLines("Задание1(вариант
сбора).txt");
        StreamWriter f1 = new StreamWriter("Задание1(вариант
сбора).txt", false);
        for (int k = 0; k < stroka1.Length; k++)
        {
            stroka1[k] = "0";
            f1.WriteLine(stroka1[k]);
        }
        f1.Close();
        string[] stroka2 = File.ReadAllLines("Звёзды.txt");
        stroka2[0] = "0";
        StreamWriter f2 = new StreamWriter("Звёзды.txt", false);
        for (int k = 0; k < stroka2.Length; k++)
        {
            f2.WriteLine(stroka2[k]);
        }
        f2.Close();
        string[] stroka3 = File.ReadAllLines("Переход к заданию.txt");
        StreamWriter f3 = new StreamWriter("Переход к заданию.txt",
false);
        for (int k = 0; k < stroka3.Length; k++)
        {
            stroka3[k] = "0";
            f3.WriteLine(stroka3[k]);
        }
        f3.Close();
        string[] stroka4 = File.ReadAllLines("Персонаж.txt");
        StreamWriter f4 = new StreamWriter("Персонаж.txt", false);
        for (int k = 0; k < stroka4.Length; k++)
        {
            stroka4[k] = "0";
            f4.WriteLine(stroka4[k]);
        }
        f4.Close();
        string[] stroka5 = File.ReadAllLines("процент выполнения
задания.txt");
        StreamWriter f5 = new StreamWriter("процент выполнения
задания.txt", false);
        for (int k = 0; k < stroka5.Length; k++)
        {
            stroka5[k] = "0";
            f5.WriteLine(stroka5[k]);
        }
        f5.Close();
        string[] stroka6 = File.ReadAllLines("сбор пазла.txt");
        StreamWriter f6 = new StreamWriter("сбор пазла.txt", false);
        for (int k = 0; k < stroka6.Length; k++)
        {
            stroka6[k] = "0";
            f6.WriteLine(stroka6[k]);
        }
        f6.Close();
        string[] stroka7 = File.ReadAllLines("форма_инф_игрока.txt");
        stroka7[0] = "";
        stroka7[1] = "0";
        stroka7[2] = "-1";
        StreamWriter f7 = new StreamWriter("форма_инф_игрока.txt",
false);
        for (int k = 0; k < stroka7.Length; k++)
        {
            f7.WriteLine(stroka7[k]);
        }
        f7.Close();
    }
    public void Method()
```

```
    {
        throw new System.NotImplementedException();
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System.Threading.Tasks;
public class playerControler : MonoBehaviour
{
    public float speed;
    public float jumpForce;
    public float moveInput;
    private Rigidbody2D rb;
    private bool facingRight=true;
    private bool isGrounded;
    public Transform freePos;
    public float checkRadius;
    public LayerMask whatIsGround;
    private Animator anim;
    private int field;
    private void Start()
    {
        anim = GetComponent<Animator>();
        rb = GetComponent<Rigidbody2D>();
    }
    private void FixedUpdate()
    {
        moveInput = Input.GetAxis("Horizontal");
        rb.velocity = new Vector2(moveInput*speed, rb.velocity.y);
        if (facingRight==false  && moveInput>0 )
        { Flip(); }
        else if (facingRight==true && moveInput<0 ) { Flip(); }
        if(moveInput==0 ) {
            anim.SetBool("isRunning", false);
        }
        else
        {
            anim.SetBool("isRunning", true);
        }
    }
    private void Update()
    {
        isGrounded = Physics2D.OverlapCircle(freePos.position,
checkRadius, whatIsGround);
        if (isGrounded == true &&
((Input.GetKeyDown(KeyCode.Space)) ||
(Input.GetKeyDown(KeyCode.UpArrow)) ||
(Input.GetKeyDown(KeyCode.W))))
        {
            rb.velocity = Vector2.up * jumpForce;
            anim.SetTrigger("taceOf");
        }
        if (isGrounded==true)
        {
            anim.SetBool("isJumping",false);
        }
        else
        {
            anim.SetBool("isJumping", true);
        }
        if (isGrounded == true &&
((Input.GetKeyDown(KeyCode.DownArrow) ||
(Input.GetKeyDown(KeyCode.S)))))
        {
            anim.SetBool("isPrised", true);
        }
        else { anim.SetBool("isPrised", false); }
    }
    void Flip()
    {
        facingRight=!facingRight;
        Vector3 Scaler = transform.localScale;
        Scaler.x *= -1;
        transform.localScale = Scaler;
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
public class MusicSelectionUI : MonoBehaviour
{
    private MusicManager musicManager;
    private int field;
    private void Start()
    {
        musicManager = FindObjectOfType<MusicManager>();
    }
    public void SelectMusic(int index)
    {
        if (musicManager != null)
        {
            musicManager.PlayMusic(index);
        }
    }
    public void TogglePause()
    {
        if (musicManager != null)
        {
            musicManager.TogglePause();
        }
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }
}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class MusicManager : MonoBehaviour
{
    public AudioClip[] musicClips;
    public AudioSource audioSource;
    private int field;
    private void Awake()
    {
        if (FindObjectsOfType<MusicManager>().Length > 1)
        {
            Destroy(gameObject);
            return;
        }
        DontDestroyOnLoad(gameObject);
        audioSource = gameObject.AddComponent<AudioSource>();
        audioSource.loop = true;
        PlayMusic(0);}
    public void PlayMusic(int index)
    {
        if (index < musicClips.Length)
        {
```

```
            audioSource.clip = musicClips[index];
            audioSource.Play();
        } }
    public void SetVolume(float volume)
    {
        audioSource=GetComponent<AudioSource>();
        audioSource.volume = volume;
    }
    public void TogglePause()
    {
        if (audioSource.isPlaying) audioSource.Pause();
            else audioSource.UnPause();
    }
    public void Method()
    {
        throw new System.NotImplementedException();
    }}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
public class LoadingScreen1 : MonoBehaviour
{
    public Slider progressBar;
    public string sceneToLoad;

    private void Start()
    {
        StartCoroutine(LoadSceneAsync() );
    }
    IEnumerator LoadSceneAsync()
    {
        AsyncOperation operation = SceneManager.LoadSceneAsync(
sceneToLoad );
        while (!operation.isDone)
        {
            float progress = Mathf.Clamp01( operation.progress/0.9f);
            progressBar.value = progress;
            yield return null;
        }}}

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class Exit : MonoBehaviour
{
    public void ExitGame()
    {
        Application.Quit();
    }
}
public void SceneManag(string sceneName)
 { SceneManager.LoadScene(sceneName); }
 public void ExitGame()
 { Application.Quit(); }
 public void LoadFileNum(string num)
 { File.WriteAllText("level.txt", num); }

using System.Collections;
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using UnityEngine.SceneManagement;

public class Переход_с_записью : MonoBehaviour
{
    private Animator anim;
    public int levelToLoad;
    private void Start()
    {
        anim = GetComponent<Animator>();
    }

    public void FadeToLevel()
    {
        anim.SetTrigger("fade");

        //получение индекса
        Scene currentScene = SceneManager.GetActiveScene();
        int sceneIndex = currentScene.buildIndex;
        StreamWriter f = new StreamWriter("Номер сцены.txt", false);
        f.WriteLine(sceneIndex.ToString());
        f.Close();
    }

    public void OnFadeComplete()
    {

        SceneManager.LoadScene(levelToLoad);
    }
}

using System.Collections;
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using UnityEngine.SceneManagement;

public class Переход_из : MonoBehaviour
{
    private Animator anim;

    private void Start()
    {
        anim = GetComponent<Animator>();
    }

    public void FadeToLevel()
    {
        anim.SetTrigger("fade");
    }

    public void OnFadeComplete()
    {
        //считывание номера сцены из файла
        StreamReader f = new StreamReader("Номер сцены.txt");
        int n = int.Parse(f.ReadToEnd());
        f.Close();
        SceneManager.LoadScene(n);
    }
}
```